

Day 0 - Setup (before Day 1)

> Time commitment: about an hour, any time before Day 1 starts. Nothing here is turned in or evaluated, but every day of the gauntlet assumes these prerequisites are already in place. Don't skip them.

What you're setting up

- A C++ compiler with C++17 support (g++ or clang; the pages just say `c++`)
- make and git
- A leak detector: `valgrind` on Linux, `leaks` on macOS. On Windows you get these through WSL.
- Catch2, the test framework Day 7 uses. Nothing to install: it's two files you drop into your project.
- Any editor you like. VS Code with the C/C++ extension works well.

Installation

Windows

Install WSL2, then follow all the Linux steps below.

In PowerShell, install the Windows Subsystem for Linux:

```
wsl --install -d Ubuntu
```

The gauntlet's flags, Makefiles, and valgrind all assume a Unix toolchain, and MSVC behaves differently in ways that will cost you time. Keep your work inside the WSL filesystem (your Linux home directory, not `/mnt/c/...`).

You may need to restart your machine after installing WSL. Once it's up, open the WSL terminal and continue with the **Linux** steps below.

WSL or Linux (Ubuntu / Debian)

```
sudo apt update
sudo apt install -y build-essential valgrind git
```

`build-essential` brings g++ and make. On other distros: `sudo dnf install gcc-c++ make valgrind git` (Fedora) or `sudo pacman -S base-devel valgrind git` (Arch).

macOS

```
xcode-select --install
```

That one command installs Apple clang, make, git, and the `leaks` tool. If it tells you the tools are already installed, you're done.

Two things to know about macOS:

- valgrind does not work on modern macOS. Use `leaks`, which the command above already installed.
- AddressSanitizer on Apple clang does not detect leaks. It's good for use-after-free, but for the gauntlet's zero-leak checks, use `leaks`.

Check 1 - the compiler, with the gauntlet's exact flags

Every exercise all week compiles with these flags, so verify with them now.
Create `hello.cpp`:

```cpp
#include <iostream>

int main(){
    std::cout << "hello, gauntlet\n";
    return 0;
}
```

Build and run:

```
c++ -Wall -Wextra -Werror -std=c++17 hello.cpp -o hello
./hello
```

If that prints `hello, gauntlet`, your compiler is ready.

Check 2 - the leak detector actually detects

Don't wait until Day 2 to find out your detector doesn't run. Create `leakcheck.cpp`, which leaks on purpose:

```cpp
// leakcheck.cpp: leaks on purpose, so you can verify your detector catches
it
#include <iostream>

int main(){
    int* forgotten = new int[10];   // never deleted: 40 bytes leak
    forgotten[0] = 42;
    std::cout << "this program leaks " << 10 * sizeof(int) << " bytes on
purpose\n";
```

```
    return 0;
}
```

Build it, then run it under your platform's detector:

```
c++ -Wall -Wextra -Werror -std=c++17 leakcheck.cpp -o leakcheck

# Linux
valgrind --leak-check=full ./leakcheck
# expect: "definitely lost: 40 bytes in 1 blocks"

# macOS
leaks --atExit -- ./leakcheck
# expect a line like: "Process NNN: 1 leak for 48 total leaked bytes."
# (the byte count may be rounded up by the allocator; the "1 leak" is what
matters)
```

If the detector reports the leak, it works. Now fix the program: add `delete[] forgotten;` before `return`, rebuild, rerun the detector, and confirm it reports 0 leaks. That clean report is what you're aiming at all week.

The Makefile you'll write every day

Every exercise folder (`ex00/`, `ex01/`, ...) ships its own Makefile with four targets: `all` (build), `clean` (remove object files), `fclean` (remove objects *and* the binary), and `re` (rebuild from scratch). Here is a canonical one. Read it until you understand it, then adapt `NAME` and `SRCS` per exercise:

```
# The binary this exercise builds
NAME := hello

# Compiler and the gauntlet's mandatory flags
CXX      := c++
CXXFLAGS := -Wall -Wextra -Werror -std=c++17

# List every .cpp file in the exercise here
SRCS := main.cpp
OBJS := $(SRCS:.cpp=.o)

# `make` / `make all`: build the binary
all: $(NAME)

$(NAME): $(OBJS)
  $(CXX) $(CXXFLAGS) $(OBJS) -o $(NAME)

# Compile each .cpp into a .o
%.o: %.cpp
  $(CXX) $(CXXFLAGS) -c $< -o $@

# `make clean`: remove object files
clean:
  rm -f $(OBJS)

# `make fclean`: remove objects AND the binary
fclean: clean
  rm -f $(NAME)
```

```
# `make re`: rebuild everything from scratch
re: fclean all

.PHONY: all clean fclean re
```

The classic gotcha: recipe lines (the indented commands) must start with a real tab character, not spaces. If make says `*** missing separator`, that's what happened, so check that your editor isn't converting tabs to spaces in Makefiles.

Try it now. Put `hello.cpp` from Check 1 in a folder as `main.cpp`, save the Makefile above next to it as `Makefile`, and confirm all four targets work:

```
make        # builds ./hello
make clean  # removes main.o
make fclean # removes main.o and hello
make re     # full rebuild
```

Get Catch2 (for Day 7)

Day 7 uses Catch2. There's no install and no CMake. The project publishes an *amalgamated* build: the whole framework flattened into one header and one source file. Grab both now while you're set up:

```
curl -L -O
https://github.com/catchorg/Catch2/releases/download/v3.12.0/catch_amalgamated.hpp
curl -L -O
https://github.com/catchorg/Catch2/releases/download/v3.12.0/catch_amalgamated.cpp
```

Keep them together somewhere you'll find again on Day 7; you'll drop a copy of both next to your test files. The `.hpp` is what your tests `#include`; the `.cpp` is the framework itself and goes in your Makefile's `SRCS`, right alongside your own sources.

Git, configured

Make sure commits carry your name:

```
git --version
git config --global user.name  "Your Name"
git config --global user.email "you@example.com"
```

You also need a GitHub account. Every training day is turned in as its own **public** repository named `gauntlet-day-01`, `gauntlet-day-02`, and so on: you push your exercises there and hand the link to your evaluator when the evaluation starts. Nothing to create today, just have the account ready.

Done when

- [ ] `hello.cpp` builds with `c++ -Wall -Wextra -Werror -std=c++17` and runs.
- [ ] Your leak detector reports the deliberate leak in `leakcheck.cpp`, and a clean report once you fix it.
- [ ] `make`, `make clean`, `make fclean`, and `make re` all work in your test folder.
- [ ] `catch_amalgamated.hpp` **and** `catch_amalgamated.cpp` are downloaded, together, somewhere you'll find on Day 7.
- [ ] `git` commits with your name and email.

Nothing to turn in. If a check fails and you can't fix it, ask staff before Day 01.